Computer Programming (CS F111)
Comprehensive Exam (Regular – Open Book Exam)
First Semester: 2024-25

Instructors: Dr. Asish Bera, Dr. Bharat Richhariya, Dr. Vinti Agarwal & Dr. Tanmaya Mahapatra (IC)
Department of Computer Science and Information Systems

NAME AND ID: _____

Important Information:

- **Write your name and ID legibly on the question paper.**
- Please check the completeness of the exam booklet **(4 double-sided pages)**
- Use only **blue** or **black** pen to answer the questions; Pencils, green and red pens are not allowed!
- Working time is **60 minutes** for **2 questions** of **30 points!**
- On any attempt to **cheat**: the exam will be assessed with a **0** and **you would be barred from makeup.**
- Permitted writing aids: T1 and printed slides + lab sheets — **open book exam!**

---

1. **The given C code implements a singly linked list in which data of each node represents a character of a given string. Next, it removes a node if two consecutive nodes have the same data. After that, reverse the list and display the modified list. Sample execution is given for easier understanding of code. Fill in the blanks with appropriate code in the EXACT LINE NUMBERs, given in code snippets in C language.** [15 pts]

Question 1: 15 pts

Expected output

```
string is: committee
Elements in the list are:
c o m m i t t e e
Delete Duplicate Nodes
Elements in the list are:
c o m i t e
Reversed List
Elements in the list are:
e t i m o c
```

```
1         #include<stdlib.h>
2         #include <stdio.h>
3         typedef struct node *NODE;
4         struct node {
5             char c;
6             NODE next;
7         };
8
9         NODE createNewNode(char data)
10        {
11            NODE nu;
12            nu=(NODE)malloc(sizeof(struct node));
13            ________________________________// [0.5 points]
14            ________________________________// [0.5 points]
15            return nu;
16        }
17
18        void insertNode(NODE p, NODE k)
19        {
20            if(k->next ==NULL) {
21                ____________________________________// [0.5 points]
22                ____________________________________// [0.5 points]
23            }
24            else {
25                NODE t = k;
26                while(t->next!=NULL)
27                t = t->next;
28                ________________________________// [0.5 points]
29                ________________________________// [0.5 points]
30            }
31        }
32
33        void ReverseList(NODE k)
34        {
35            NODE karent = k->next;
36            NODE prev = NULL; NODE aftr = NULL;
```

```
            while (karent != NULL) {
                ____________________________________________// [1 point]
                ____________________________________________// [1 point]
                ____________________________________________// [1 point]
                ____________________________________________// [1 point]
            }
            ____________________________________________// [1 point]
        }

        void removeDuplicate(NODE start)
        {
            NODE karent = start;
            NODE t;

            if (karent == NULL)
            return;

            while (karent->next != NULL)
            {
                if (_____________________________)// [1 point]
                {
                    ________________________________// [1 point]
                    ________________________________// [1 point]
                    ________________________________// [1 point]
                }
                else
                ________________________________// [1 point]
            }
        }

        void display(NODE p) {
            NODE t = p->next;
            if (t == NULL)
            printf("List is empty.\n");
            printf("Elements in the list are: \n") ;
            while (_____________________________); // [0.5 point]
            {   printf(" %c \t", _________________); // [0.5 point]
                t = t->next;
            }
        }

        int main(){
            NODE start= (NODE)malloc(sizeof(struct node));
            start->next=NULL;
            char *str="committee";
            printf("string is: %s\n", _____________________);// [0.5 point]
            while(*str!='\0')
            {
                NODE N=createNewNode(____________________); // [0.5 point]
                insertNode(N,start);
            }
            display(start);

            printf("\n Delete Duplicate Nodes\n");
            removeDuplicate(start);
            display(start);

            printf("\nReversed List\n");
            ReverseList(start);
            display(start);
            return 0;
        }
```

2. **MinMaxSort() is a C function that sorts an array *arr[ ]* of n integers in ascending order. It achieves this by finding both the minimum and maximum elements of the working array *arr[ ]*, storing them in a new array *sortArr[ ]* at their respective left and right locations, and replacing the min and max elements in *arr[ ]* with -1. This process is repeated iteratively until *sortArr[ ]* is not entirely filed.** [15 pts]

   This is done using the *findMinIndex( )* and *findMaxIndex( )* functions. The elements in the new array are sorted using *MinMaxSort( )* function.

   The entire working array *arr[ ]* is scanned at every iteration to find the minimum and maximum elements, except those with -1 values. The process continues until *sortArr[ ]* is not entirely filled.

   Fill up the gaps in the below code: You are not required to introduce any new variables.

```
            #include<stdio.h>
            #include <limits.h>

            int findMinIndex(int arr[], int size) {
                int min = __INT_MAX__;  // Initialize min value to maximum possible integer value
```

```
6               int minId = -1;  // Initialize min INDEX  to unexpected integer value -1
7               for (int i = 0; i < size; ++i) {
8                   if (___Write your code here______)        // [2 points]
9                   {
10                      ____Write your code here_____           // [3 points]
11                  }
12              }
13              return minId;
14          }
15
16          int findMaxIndex(int arr[], int size) {
17              int max = INT_MIN; // Initialize min to maximum possible integer value
18              int maxId = -1 ;  // Initialize max INDEX  to unexpected integer value -1
19              for (int i = 0; i < size; ++i) {
20                  if (___Write your code here______)   // [2 points]
21                  {
22                      ____Write your code here_____           // [3 points]
23                  }
24
25              }
26              return maxId;
27          }
28
29          /* Sorting an array with n elements in ascending order */
30          void MinMaxSort ( int arr[], int n ) {
31              int minId, maxId; // declaration of min index, max index of working array
32              sortArr[n];           //  // declaration of sorted array
33              int left = 0;       // holds the leftmost index of the sorted array
34              int right = n-1;        // holds the rightmost index of the sorted array
35              while (___Write your code here______ ) // To ensure sorted array is not entirely
                    filled  [01 point]
36              {
37                  minId = findMinIndex( arr, n );
38                  maxId = findMaxIndex( arr, n);
39                  ___Write your code here___ // To store the min and max elements from working to
                        sorted array [02 points]
40                  arr[minId] = -1;
41                  arr[maxId] = -1;
42                  ___Write your code here___ // To update left and right index of sorted array
                        [02 points]
43              }
44              printf("\nSorted array is\n");
45              for (int i = 0; i < n; i++)
46              printf("%d \t", sortArr[i]);
47          }
48
49          int main()
50          {
51              int arr[] = {3, 5, 112, 8, 10, 90};
52              int n=sizeof(arr) / sizeof(arr[0]);
53              MinMaxSort(arr, n);
54          }
```

Question 2: 15 pts

****

(Don't let fear get the better of you. All the best!)

Space for Rough Work